

SIMATS ENGINEERING

ASSESSMENT TOOL 2 - MEDIUM (Assignment)

COURSE CODE: CSA09

COURSE NAME: Programming in Java

COURSE OUTCOME COVERED

CO2: *Implement Collection Framework interfaces such as List, ArrayList, Set, Map, HashTable, and HashMap using iterators and generics to store, retrieve, and process groups of objects in web applications. (BL3,BL4)*

Assessment Tool Weightage: 20%

QUESTIONS: 3

Total Marks: 30

ANNEXURE A

Assignment

Code of Conduct:

*I **O. BHANU PRAKASH REDDY - 192372075** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: obhanuprakashredddy

ANNEXURE

Assignment

1. ATM PIN Security Validator

"A nationalized bank receives ATM PINs from customers through a kiosk. Develop a Java program that accepts the PIN as a string and performs the following operations:

Remove unwanted spaces using trim()

Validate that the PIN contains exactly 4 digits using matches()

Display the PIN length using length()"

Code:

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter ATM PIN: ");
        String pin = sc.nextLine();

        pin = pin.trim();

        if (pin.matches("\\d{4}")) {
            System.out.println("Valid PIN");
        } else {
            System.out.println("Invalid PIN");
        }

        System.out.println("PIN Length: " + pin.length());
    }
}
```

Input:

```
Enter ATM PIN: 1234
```

Output:

```
Enter ATM PIN: 1234
Valid PIN
PIN Length: 4
```

2. Employee ID Normalization System

"An HR department receives employee IDs in mixed formats such as emp1023.

Create a Java program to:

Remove spaces using trim()

Convert the alphabetic prefix to uppercase using toUpperCase()

Join the prefix and number using concat()

Code:

```
import java.util.Scanner;

public class Employee_ID {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter Employee ID: ");
        String id = sc.nextLine().trim();

        String prefix = "";
        String number = "";

        for (int i = 0; i < id.length(); i++) {
            if (Character.isLetter(id.charAt(i)))
                prefix += id.charAt(i);
            else
                number += id.charAt(i);
        }

        prefix = prefix.toUpperCase();

        String result = prefix.concat(number);

        System.out.println("Normalized ID: " + result);
    }
}
```

Input:

```
emp1023
```

Output:

```
Enter Employee ID: emp1023
Normalized ID: EMP1023
```

3. Online Shopping Coupon Verificatio

"An e-commerce website offers the coupon code SAVE50. Write a Java program that:

Removes extra spaces using trim()

Converts the entered code to uppercase using toUpperCase()

Compares it with the original coupon using equalsIgnoreCase()"

Code:

```
import java.util.Scanner;

public class Coupon {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String original = "SAVE50";

        System.out.print("Enter Coupon Code: ");
        String code = sc.nextLine();

        code = code.trim();
        code = code.toUpperCase();

        if (code.equalsIgnoreCase(original))
            System.out.println("Coupon Applied");
        else
            System.out.println("Invalid Coupon");
    }
}
```

Input:

```
save50
```

Output:

```
Enter Coupon Code: save50  
Coupon Applied
```

4. Student Email Username Extractor

"A college portal stores student emails such as arun23@college.edu.

Develop a program that:

Finds the position of @ using indexOf()

Extracts the username using substring()

Displays the username length using length()

Code:

```
import java.util.Scanner;  
  
public class Student_Email {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
  
        System.out.print("Enter Email: ");  
        String email = sc.nextLine();  
  
        int pos = email.indexOf("@");  
  
        String username = email.substring(0, pos);  
  
        System.out.println("Username: " + username);  
        System.out.println("Username Length: " + username.length());  
    }  
}
```

Output:

```
arun23@college.edu
```

Input:

```
Enter Email: arun23@college.edu  
Username: arun23  
Username Length: 6
```

5. Restaurant Digital Order Analyzer

"A restaurant stores orders as DOSA-3-IDLY-2-VADA-1. Create a

program that:

Splits the order using split()

Converts food names to uppercase using toUpperCase()
Counts the number of ordered items using length()

Code:

```
import java.util.Scanner;

public class Restaurant_Order {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter Order: ");
        String order = sc.nextLine();

        String[] items = order.split("-");

        System.out.println("Food Items:");

        int count = 0;

        for (int i = 0; i < items.length; i += 2) {
            System.out.println(items[i].toUpperCase() + " Quantity: " + items[i + 1]);
            count++;
        }

        System.out.println("Total Food Items: " + count);
    }
}
```

Input:

```
DOSA-3-IDLY-2-VADA-1
```

Output:

```
Enter Order: DOSA-3-IDLY-2-VADA-1
Food Items:
DOSA Quantity: 3
IDLY Quantity: 2
VADA Quantity: 1
Total Food Items: 3
```

6. Railway Coach and Seat Parser

"Indian Railways stores ticket information as S5-43. Write a Java program that:

Splits the value using split()

Extracts the coach number using substring()

Displays the coach type using charAt()

Code:

```
import java.util.Scanner;
```

```
public class RailwayCoach {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
  
        System.out.print("Enter Coach and Seat (Ex: S5-43): ");  
        String ticket = sc.nextLine();  
  
        String[] parts = ticket.split("-");  
  
        String coach = parts[0];  
        String seat = parts[1];  
  
        System.out.println("Coach: " + coach);  
        System.out.println("Seat Number: " + seat);  
        System.out.println("Coach Number: " + coach.substring(1));  
        System.out.println("Coach Type: " + coach.charAt(0));  
    }  
}
```

Input:

```
S5-43
```

Output:

```
Enter Coach and Seat (Ex: S5-43): S5-43  
Coach: S5  
Seat Number: 43  
Coach Number: 5  
Coach Type: S
```

7. Inventory Product Search System

"A supermarket stores product names in different letter cases. Develop a program that:

Converts product names to lowercase using toLowerCase()

Searches a product using contains()

Confirms exact matching using equalsIgnoreCase()

Code:

```
import java.util.Scanner;
```

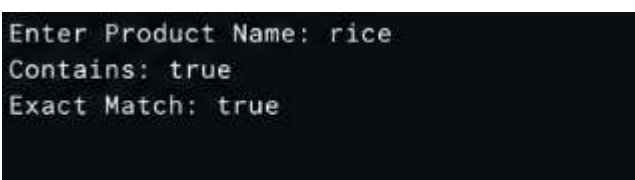
```
public class ProductSearch {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
  
        String product = "Rice";  
  
        System.out.print("Enter Product Name: ");  
        String input = sc.nextLine();  
  
        String p1 = product.toLowerCase();  
        String p2 = input.toLowerCase();  
  
        System.out.println("Contains: " + p1.contains(p2));  
        System.out.println("Exact Match: " + product.equalsIgnoreCase(input));  
    }  
}
```

Input:



```
rice
```

Output:



```
Enter Product Name: rice  
Contains: true  
Exact Match: true
```

8. Grocery Shopping Cart Management System

"Add products using add()

Remove purchased products using remove()

Check whether a product exists using contains() Check how many products there in the list"

Code:

```
import java.util.*;
```

```
public class ShoppingCart {
```



```

public static void main(String[] args) {
    ArrayList<String> cart = new ArrayList<>();

    cart.add("Rice");
    cart.add("Sugar");
    cart.add("Oil");
    cart.add("Milk");

    System.out.println("Cart: " + cart);

    cart.remove("Oil");

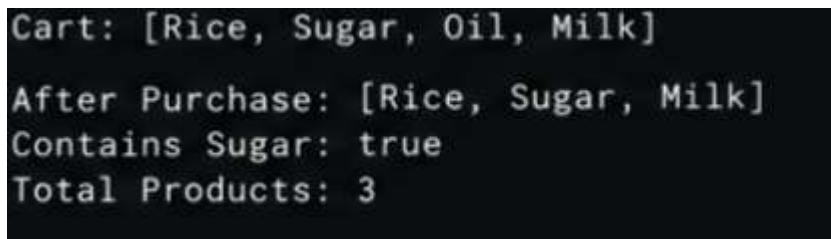
    System.out.println("After Purchase: " + cart);

    System.out.println("Contains Sugar: " + cart.contains("Sugar"));

    System.out.println("Total Products: " + cart.size());
}
}

```

Output:



```

Cart: [Rice, Sugar, Oil, Milk]
After Purchase: [Rice, Sugar, Milk]
Contains Sugar: true
Total Products: 3

```

9. Bank Token Queue Management System

"A bank serves customers based on token order. Using Queue:

Add customer tokens using offer()

Serve the next customer using poll()

Display the upcoming customer using peek()"

Code:

```

import java.util.*;

public class BankQueue {
    public static void main(String[] args) {

        Queue<Integer> queue = new LinkedList<>();

        queue.offer(101);
        queue.offer(102);
        queue.offer(103);
    }
}

```

```

        System.out.println("Queue: " + queue);

        System.out.println("Served Token: " + queue.poll());

        System.out.println("Next Token: " + queue.peek());

        System.out.println("Remaining Queue: " + queue);
    }
}

```

Output:

```

Queue: [101, 102, 103]
Served Token: 101
Next Token: 102
Remaining Queue: [102, 103]

```

10. Railway Parcel Dispatch Tracking System

"A railway parcel office loads parcels onto a dispatch container. The last loaded parcel must be unloaded first, and parcel details must be searchable.

Requirements:

Store parcel IDs in a Stack

Store parcel ID and destination in a HashMap

Unload parcels using pop() and display the destination of the unloaded parcel"

Code:

```

import java.util.*;

public class ParcelTracking {
    public static void main(String[] args) {

        Stack<String> stack = new Stack<>();
        HashMap<String, String> map = new HashMap<>();

        stack.push("P101");
        stack.push("P102");
    }
}

```

```

stack.push("P103");

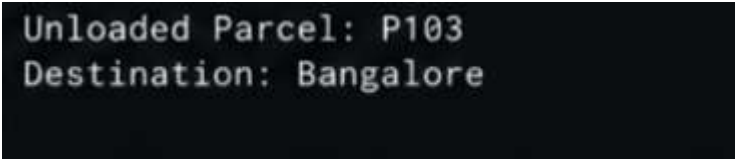
map.put("P101", "Chennai");
map.put("P102", "Hyderabad");
map.put("P103", "Bangalore");

String parcel = stack.pop();

System.out.println("Unloaded Parcel: " + parcel);
System.out.println("Destination: " + map.get(parcel));
}
}

```

Output:



```

Unloaded Parcel: P103
Destination: Bangalore

```

11. Hospital Appointment Scheduling System

"A hospital issues appointment tokens to patients and stores patient details separately.

Requirements:

Add appointment tokens to a Queue

Store token number and patient name in a HashMap

Serve patients in token order and display the served patient details"

Code:

```

import java.util.*;

public class HospitalAppointment {
    public static void main(String[] args) {

        Queue<Integer> queue = new LinkedList<>();
        HashMap<Integer, String> patient = new HashMap<>();

        queue.offer(1);
        queue.offer(2);
        queue.offer(3);

        patient.put(1, "Arun");
        patient.put(2, "Bhanu");
        patient.put(3, "Kiran");
    }
}

```

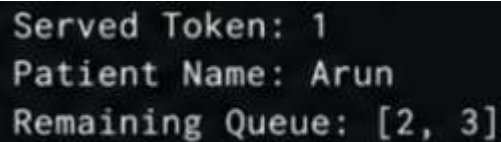
```

        int token = queue.poll();

        System.out.println("Served Token: " + token);
        System.out.println("Patient Name: " + patient.get(token));
        System.out.println("Remaining Queue: " + queue);
    }
}

```

Output



```

Served Token: 1
Patient Name: Arun
Remaining Queue: [2, 3]

```

12. University Course Enrollment Manager

"A university maintains a list of students enrolled in a course. Students who withdraw must be removed from the list.

Requirements:

Add student names to an ArrayList

Traverse the list using an Iterator

Remove withdrawn students and display the updated enrollment list"

Code:

```

import java.util.*;

public class CourseEnrollment {
    public static void main(String[] args) {

        ArrayList<String> students = new ArrayList<>();

        students.add("Arun");
        students.add("Bhanu");
        students.add("Kiran");
        students.add("Ravi");

        Iterator<String> it = students.iterator();

        while (it.hasNext()) {
            String name = it.next();
            if (name.equals("Kiran")) {
                it.remove();
            }
        }
    }
}

```

```

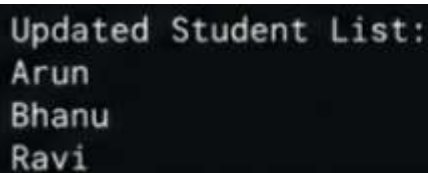
    }
}

System.out.println("Updated Student List:");

for (String s : students)
    System.out.println(s);
}
}

```

Output:



```

Updated Student List:
Arun
Bhanu
Ravi

```

13. Warehouse Damaged Item Removal

“A warehouse stores item names in an inventory list. Damaged items must be removed safely while traversing the collection.

Requirements:

Store inventory items in an ArrayList

Traverse the list using an Iterator

Remove items marked as Damaged and display the remaining inventory"

Code:

```

import java.util.*;

public class Warehouse {
    public static void main(String[] args) {

        ArrayList<String> items = new ArrayList<>();

        items.add("Rice");
        items.add("Sugar");
        items.add("Damaged");
        items.add("Oil");

        Iterator<String> it = items.iterator();

        while (it.hasNext()) {
            String item = it.next();

```

```

        if (item.equals("Damaged")) {
            it.remove();
        }
    }

    System.out.println("Remaining Inventory:");

    for (String item : items)
        System.out.println(item);
    }
}

```

Output:



```

Remaining Inventory:
Rice
Sugar
Oil

```

14. Bank Token Management System

"Customers receive tokens in the order they arrive at a bank counter.

Requirements:

Add customer tokens to a Queue

Serve customers in FIFO order

Display the next customer to be served and the remaining queue"

Code:

```

import java.util.*;

public class BankToken {
    public static void main(String[] args) {

        Queue<String> queue = new LinkedList<>();

        queue.offer("T101");
        queue.offer("T102");
        queue.offer("T103");

        System.out.println("Next Customer: " + queue.peek());

        System.out.println("Served Customer: " + queue.poll());

        System.out.println("Remaining Queue: " + queue);
    }
}

```

```
}  
}
```

Output:

```
Next Customer: T101  
Served Customer: T101  
Remaining Queue: [T102, T103]
```

15. Web Browser Back Navigation

"A web browser stores recently visited pages so that users can navigate backward.

Requirements:

Add visited page URLs using push()

Perform the Back operation using pop()

Display the current page using peek()

Code:

```
import java.util.*;  
  
public class BrowserNavigation {  
    public static void main(String[] args) {  
  
        Stack<String> pages = new Stack<>();  
  
        pages.push("google.com");  
        pages.push("youtube.com");  
        pages.push("openai.com");  
  
        System.out.println("Current Page: " + pages.peek());  
  
        System.out.println("Back To: " + pages.pop());  
  
        System.out.println("Current Page After Back: " + pages.peek());  
    }  
}
```

Output:

Current Page: openai.com
Back To: openai.com
Current Page After Back: youtube.com

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30	Applies appropriate concepts effectively to solve complex real-world problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & Decision-Making Skills	20	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach

Clarity, Justification & Presentation	10	Clear, well- justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification
---	----	---	---	--	---